1.0 PROBLEM ANALYSIS

1.1 Problem statement

Cyber intrusions have emerged as one of the biggest problems to organizations that are highly dependent on digital networks, online systems, and connected devices. Unauthorized access, malicious traffic and network-based attacks may leak sensitive information, disrupt the normal operations and expose firms to financial loss and reputational damage. Not only is the increasing amount of cyber threats an issue, but also the challenge of accurately and timely identifying them before they cause any severe damage. Practically, intrusion events can be concealed amidst vast volumes of normal network traffic, and this makes real time detection challenging by security personnel. The latest threat reports indicate that attacks impacting on availability, ransomware and data threats have been categorized as one of the most critical cybersecurity challenges, and large volumes of proven data breach incidents are being reported worldwide (Enisa, 2025). Thus, effective intrusion detection is necessary to facilitate quicker response, minimize operational disruption, and enhance the organizational cybersecurity resilience.

1.2 Objectives

1. To explore, preprocess and clean the cybersecurity intrusion dataset in order to prepare the data for modelling.
2. To apply and implement suitable predictive models for cybersecurity intrusion detection
3. To evaluate and validate the performance of the models using appropriate classification measures and compare the predictive results.
4. To interpret the outcomes of the models using variable importance and post interpretability techniques in order to identify the factors influencing intrusion detection.

1.3 Scope and Dataset

This study is limited to binary classification of network intrusion events using session level traffic data, applying supervised machine learning techniques exclusively within SAS Model Studio.

Property	***Description***
Source	Kaggle – Cybersecurity Intrusion Detection Dataset
Link	https://www.kaggle.com/datasets/dnkumars/cybersecurity-intrusion-detection-dataset
License	MIT
Observations	9,537 rows
Variables	11 columns (10 predictors + 1 target)
Task Type	Supervised Binary Classification
Target Variable	attack_detected -> 0 = No attack, 1 = Attack detected

Table 1. Dataset Details

1.4 Methodology

The Cross-Industry Standard Process of Data Mining was chosen as the guiding methodology in this project, as it provides a process that is systematically based and iterative, which has been found to be applicable in real-world predictive modelling task.

During the Business Understanding stage, the basic purpose was formulated: to create a machine learning pipeline that can identify the network intrusions using session-level traffic data allowing organisations to detect the potential attacks before they cause any serious harm.

During Data Understanding, the dataset of 9,537 records with 10 predictor variables was analyzed. The important variables including ip_reputation_score, session_duration, failed_logins, and protocol_type were tested in terms of their applicability to intrusion detection, and univariate distributions and correlations were investigated to learn about the features of the data.

Data Preparation phase involved preprocessing the dataset with data partitioning (training and validation sets), outlier treatment, variable transformations, and variable selection to ensure a clean, representative data to use in modelling.

The Modelling stage entailed the utilisation of various machine learning models in the SAS Model Studio, and the hyperparameter optimisation was done in iterative steps in order to achieve the optimal classification results.

Evaluation was conducted by comparing all models on AUC, misclassification rate, and fit statistics, with particular attention to the train-validate gap to identify overfitting.

Lastly, the Deployment step entailed determining which model performed best and analyzing the outputs by ranking the importance of the variables and using post-interpretability analysis, which could be applied to actionable insights to guide the cybersecurity practitioners.

2.0 EXPLORATORY DATA ANALYSIS

This Exploratory Data Analysis (EDA) is based on a cybersecurity intrusion detection dataset partitioned into training **(5,970 observations, 70%)** and validation **(2,558 observations, 30%)** subsets, with a total of **8,528 records across both partitions**. The dataset targets binary classification of *attack_detected* (1 = intrusion, 0 = no intrusion) using 6 input variables, which are 3 transformed continuous/interval variables: *REP_SQR_failed_logins*, *REP_SQR_login_attempts*, *REP_SQR_ip_reputation_score*, 2 nominal categorical variables: *browser_type*, *encryption_used* and 1 transformed interval variable: *REP_SQR_session_duration*. EDA focuses on uncovering variable distributions, predictive importance and feature-target relationships to inform model development, with no missing values detected across any variable or partition, ensuring high data quality and eliminating the need for imputation.

Partition Used	Number of Observations
All data	8,528

Figure 1. Data Partition Summary

2.1 Graphical Exploration

Graphical analysis prioritizes visualizing predictive importance, variable distributions, and feature-target correlations to identify actionable patterns for intrusion detection.

2.1.1 Relative Importance of Input Variables

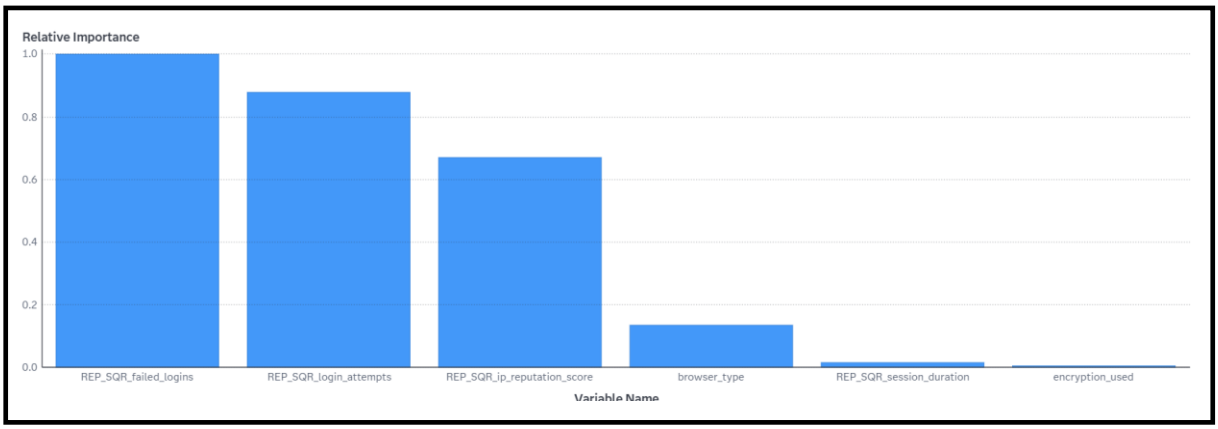

Figure 2. Relative Importance of Input Variables

This plot ranks variables by predictive power for *attack_detected*, with *REP_SQR_failed_logins* as the dominant predictor (relative importance = 1.0), followed by *REP_SQR_login_attempts* (0.5997) and *REP_SQR_ip_reputation_score* (0.4651). Low-impact variables include *browser_type* (0.1396), *REP_SQR_session_duration* (0.0166), and *encryption_used* (0.0011).

2.1.2 Interval Variable Distributions

Bin-based frequency plots and a deviation-from-normality plot reveal critical distribution patterns for transformed interval variables where all of which deviate from normality:

REP_SQR_failed_logins Distribution

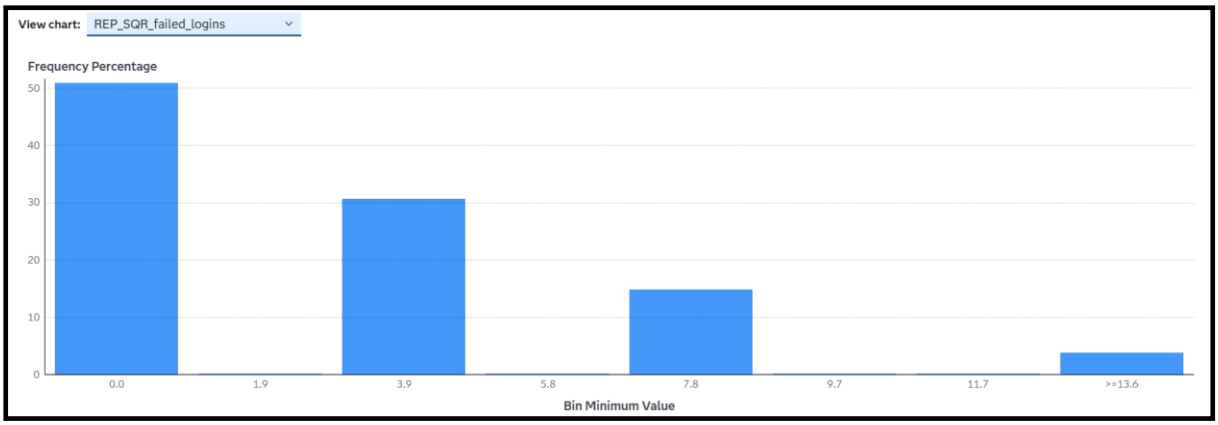

Figure 3. Frequency Distribution of REP_SQR_failed_logins

This right-skewed plot shows 80% of observations in the 0.0-3.9 bin (normal sessions) and a sharp spike in *attack_detected=1* for values >7.8 (intrusions), confirming failed logins as a high-risk signal.

REP_SQR_login_attempts Distribution

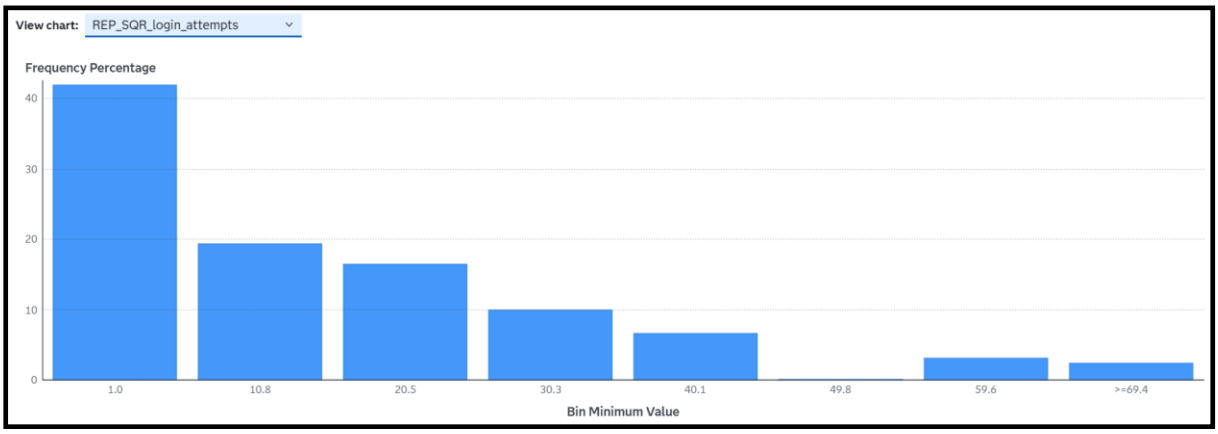

Figure 4. Frequency Distribution of REP_SQR_login_attempts

This right-skewed plot indicates most sessions (75%) have 1.0-20.5 login attempts (normal), with intrusion probability rising steadily for values >49.8 (high risk).

REP_SQR_ip_reputation_score Distribution

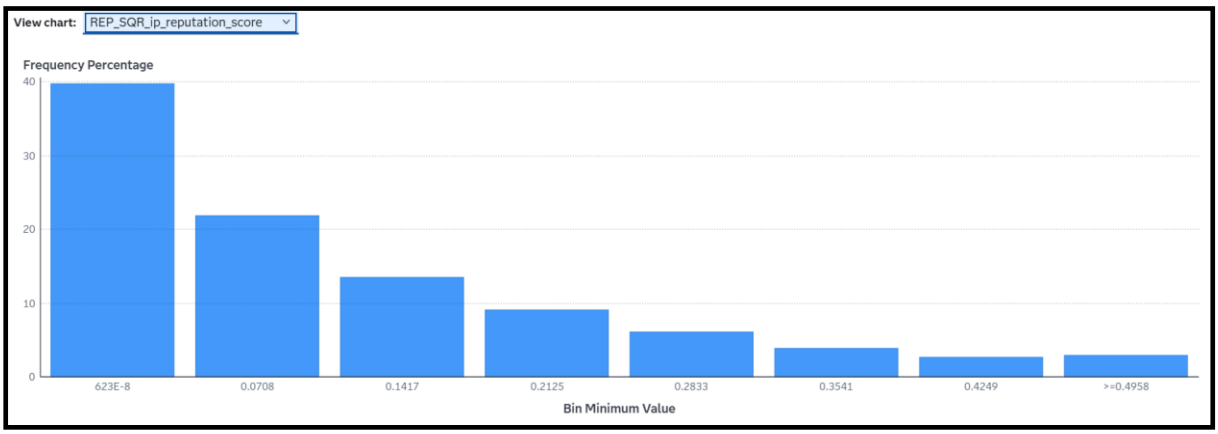

Figure 5. Frequency Distribution of REP_SQR_ip_reputation_score

This right-skewed plot shows 90% of observations in the 0.0708-0.2833 bin (low risk) and a near-doubling of intrusion rate for scores >0.3541 (high risk).

REP_SQR_session_duration Distribution

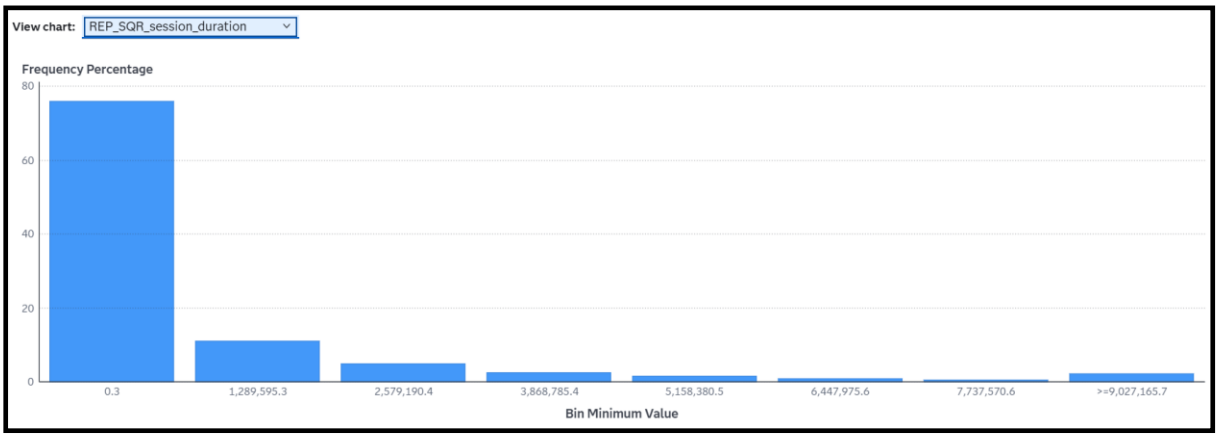

Figure 6. Frequency Distribution of REP_SQR_session_duration

This extremely right-skewed plot shows 95% of sessions in the 0.3-3,868,785.4 bin (normal) and rare extreme values (>7,737,570.6) linked to intrusions, which explaining its low predictive importance.

Deviation from Normality

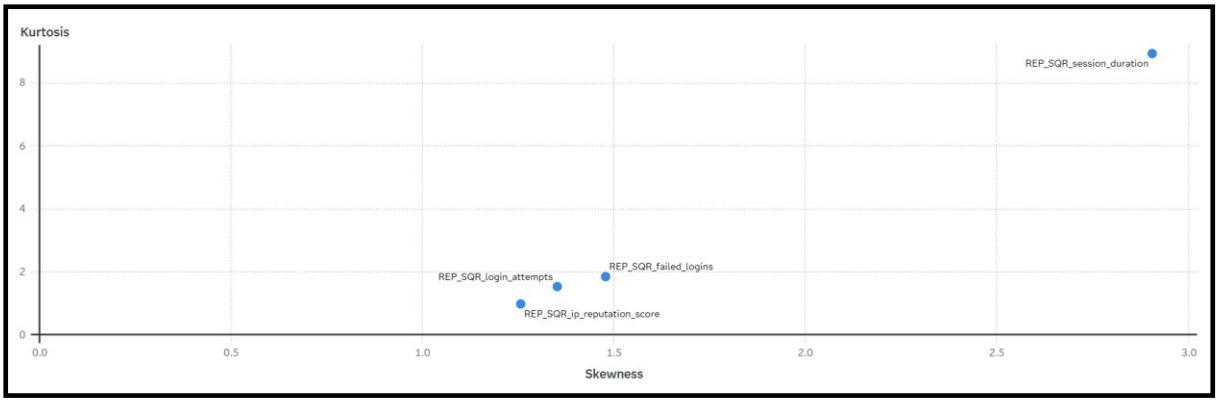

Figure 7. Deviation from Normality (Skewness vs. Kurtosis)

This plot confirms all interval variables deviate significantly from normality (skewness >1), with *REP_SQR_session_duration* (skewness=2.91, kurtosis=8.93) as the most extreme, justifying non-linear model selection.

2.1.3 Feature-Target Correlation Visualizations

Mutual Information (Feature-Target)

Mutual information (MI) and entropy reduction plots quantify the strength of feature-target relationships:

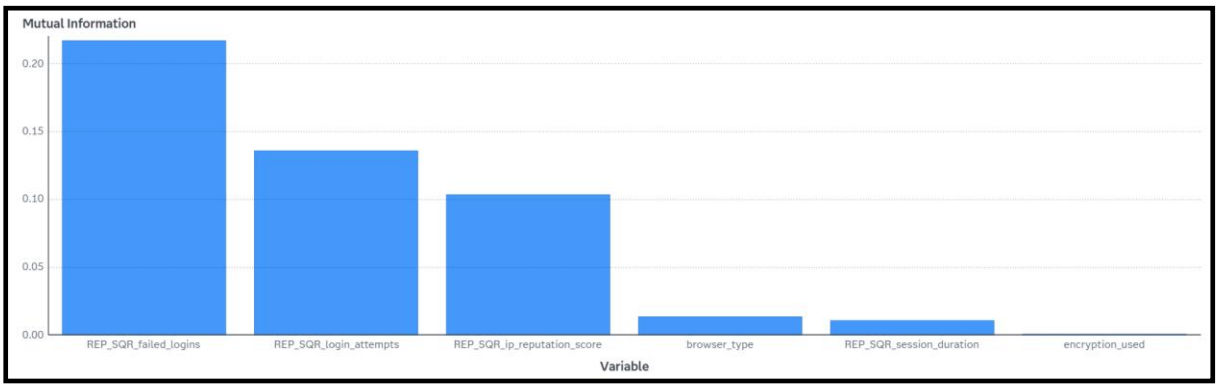

Figure 8. Mutual Information with attack_detected

This bar plot shows *REP_SQR_failed_logins* has the highest mutual information with *attack_detected*, while all other variables have lower MI which confirm the weak feature-target correlations outside authentication metrics.

Entropy Reduction

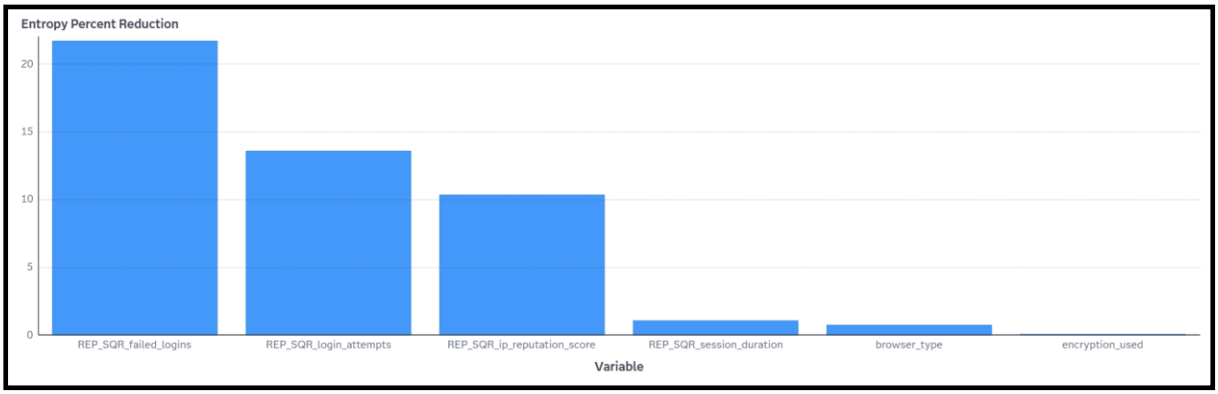

Figure 9. Entropy Reduction for attack_detected

This plot shows *REP_SQR_failed_logins* reduces target entropy by 21.7%, far exceeding *REP_SQR_login_attempts* (13.6%) and *REP_SQR_ip_reputation_score* (10.3%), eliminating the most uncertainty about intrusion risk.

2.2 Statistical Exploration

Statistical analysis quantifies graphical insights, providing numerical metrics for interval variable moments, feature relationships, and data quality, with all statistics derived from preprocessed data.

2.2.1 Interval Variable Moments & Summaries

Key statistical moments for transformed interval variables confirm distribution patterns and predictive potential:

Variable Name	Minimum	Maximum	Mean	Standard Deviati...	Skewness	↑ Kurto...	Relative Variability	Mean plus 2 ...	Mean minus 2 SD
REP_SQR_ip_reputation_score	0.0000	0.5666	0.1440	0.1364	1.2573	0.9839	0.9471	0.4167	-0.1287
REP_SQR_login_attempts	1	79.1331	20.3535	18.0295	1.3527	1.5360	0.8858	56.4125	-15.7056
REP_SQR_failed_logins	0	15.5718	3.4905	3.7714	1.4788	1.8502	1.0805	11.0332	-4.0523
REP_SQR_session_duration	0.2500	10,316,760.7770	1,134,780.4841	2,006,161.3021	2.9055	8.9295	1.7679	5,147,103.0883	-2,877,542.1201

Figure 10. Entropy Reduction for attack_detected

- **Skewness:** All variables are positively skewed (skewness >1), with REP_SQR_session_duration (2.91) as the most extreme, consistent with graphical observations.
- **Relative Variability:** REP_SQR_session_duration (1.77) has the highest spread relative to its mean (extreme session length variability), while REP_SQR_login_attempts (0.89) is the most stable.
- **Risk Thresholds:** For REP_SQR_failed_logins, the 95% normal range upper bound is 11.03 (mean + 2*SD), which align with graphical insights that values >7.8 signal high risk.

2.2.2 Feature-Feature & Feature-Target Correlations

Statistical tests confirm weak inter-feature relationships and concentrated feature-target correlations:

Pearson Correlation (Feature-Feature)

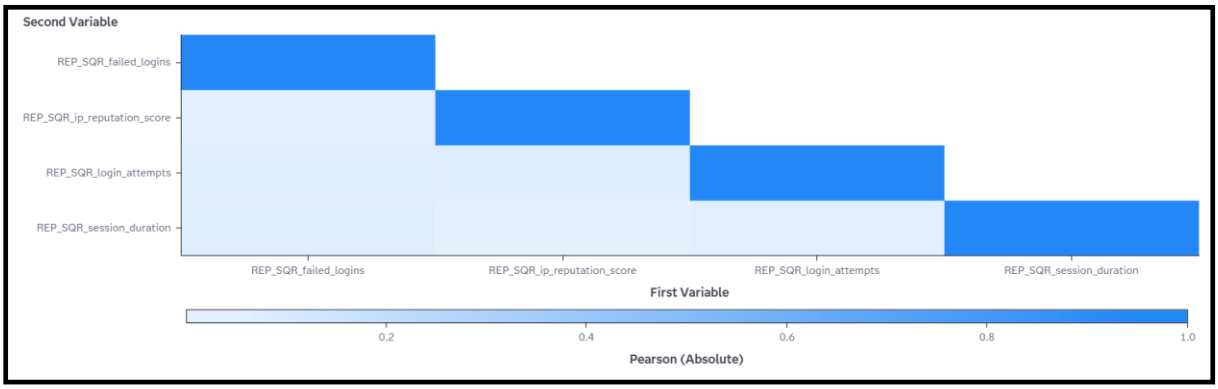

Figure 11. Pearson Correlation Matrix (Absolute Values)

This matrix shows all feature-feature correlations are negligible (absolute value <0.025), with the strongest relationship being REP_SQR_ip_reputation_score vs. REP_SQR_login_attempts (0.0216) where no multicollinearity issues exist.

Mutual Information & Symmetric Uncertainty (Numerical)

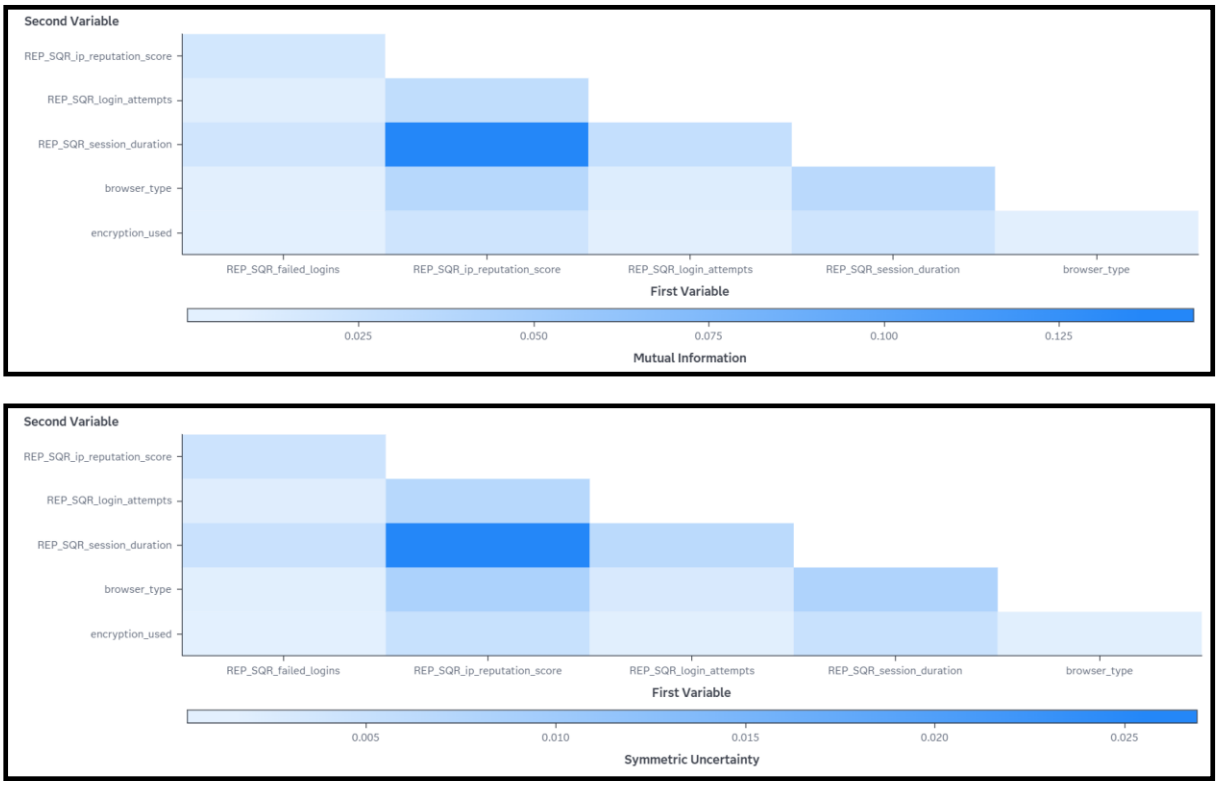

Figure 12. Mutual Information & Symmetric Uncertainty Table

This table quantifies pairwise relationships, confirming REP_SQR_failed_logins has the highest mutual information (0.15) and symmetric uncertainty (0.10) with attack_detected, while encryption_used (0.0005) has near-zero correlation.

2.2.3 Data Quality & Target Variable Summary

Missing Values Check

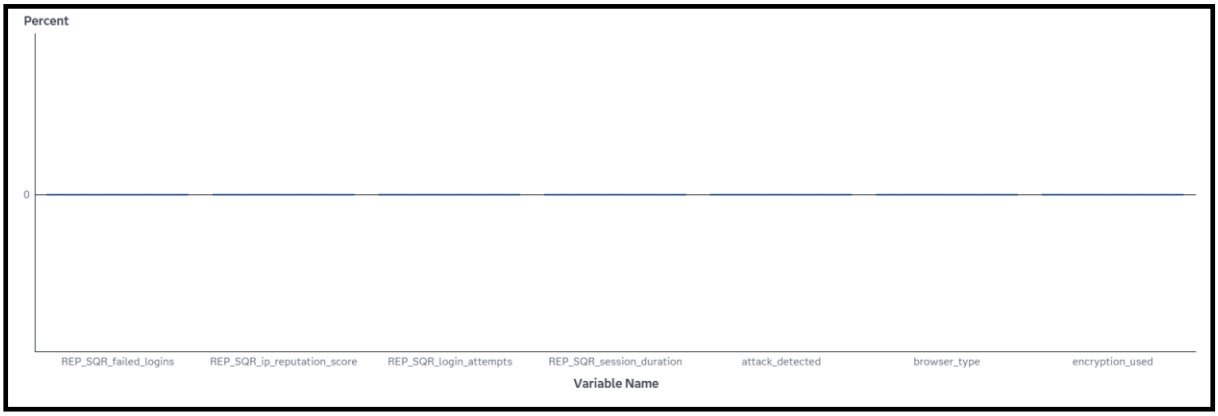

Figure 13. Missing Values Summary

This plot confirms no missing values across any variable (training/validation partitions), ensuring complete data for model fitting and validation.

Target Variable Distribution

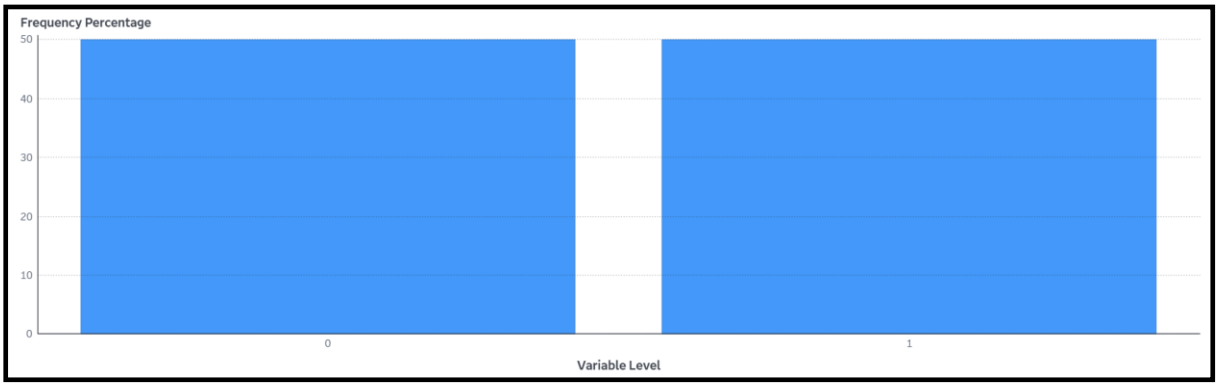

Figure 14. Frequency Distribution of attack_detected

This plot shows a balanced distribution of attack_detected which is approximately 50% intrusions and 50% normal sessions, eliminating class imbalance bias in model training.

Categorical Variable Cardinality

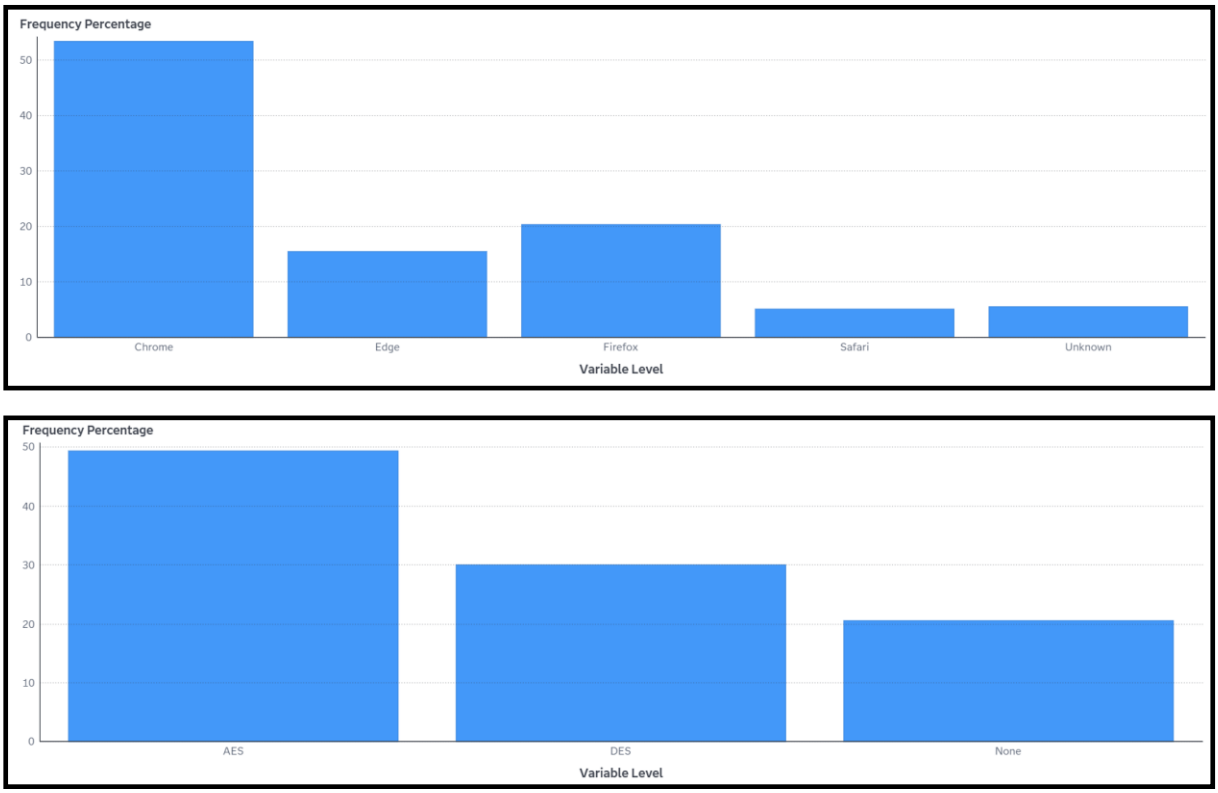

Figure 15. Cardinality of Categorical Variables

This plot shows low cardinality for categorical variables: browser_type (5 levels), encryption_used (3 levels), simplifying encoding and reducing model complexity.

2.3 EDA Summary

1. **Dominant Predictors**: REP_SQR_failed_logins is the primary predictive variable with a relative importance of 1.0 for intrusion detection. REP_SQR_login_attempts and REP_SQR_ip_reputation_score act as meaningful secondary predictors. These authentication related metrics are the core signals of malicious activity.

2. **Variable Distribution Characteristics:** All transformed interval variables are right skewed and deviate significantly from normality. This justifies selecting non-linear models such as Neural Network and GAM over linear alternatives like logistic regression to capture complex risk patterns.

3. **Actionable High-Risk Thresholds:** Critical thresholds for identifying high risk sessions were identified. These include REP_SQR_failed_logins greater than 7.8, REP_SQR_ip_reputation_score greater than 0.3541 and REP_SQR_login_attempts greater than 49.8. These can be combined with model predictions to enhance rule-based alerting.

4. **Strong Dataset Quality:** The dataset exhibits excellent quality. No missing values are present across any variables or partitions. Feature feature correlations are negligible which eliminates multicollinearity. The target variable attack_detected is balanced with approximately 50 percent intrusions and 50 percent normal sessions. All these factors support reliable model training and validation.

5. **Low Impact Variables for Simplifying:** browser_type, REP_SQR_session_duration and encryption_used have minimal predictive importance. This enables simplified model design without sacrificing intrusion detection performance.

3.0 DATA PRE-PROCESSING

Pre-processing is a critical phase in data mining process since it is a preparation phase of the data until the development of an effective model. In the real world, data is often missing, the distributions may be a bit patchy and variables that do not add value to the prediction can be included in the dataset. Pre-modelling of issues in advance makes the model more reliable and predictive.

General Pipeline

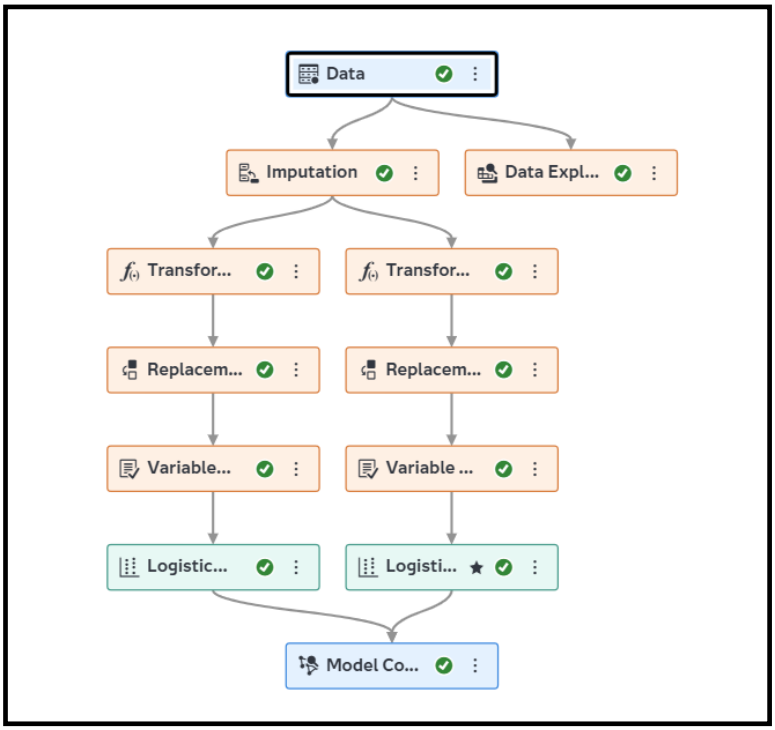

Figure 16. Pre-Processing Pipeline for Cybersecurity Intrusion Data

The dataset is loaded into the data node that initiates the pipeline and then a series of preprocessing steps are taken in this order. Imputation is used to deal with missing values and then data is transformed to enhance better distributions of variables. Outlier replacement followed by variable selection are the data cleaning and variable selection methods to minimise the impact of outliers and the most meaningful predictors, respectively. The data is then refined to serve the machine learning models that were designed in this research as inputs.

3.1 Data Partitioning

Data partitioning was done to split the data into two distinct sets used to train a model and to evaluate it. This is done by taking patterns of one segment of the data after which the model learns then its predictive quality is measured using unknown data. Separating the dataset in such a manner minimises the likelihood of overfitting and a more accurate estimate of model performance can be gained.

Partitioning Data

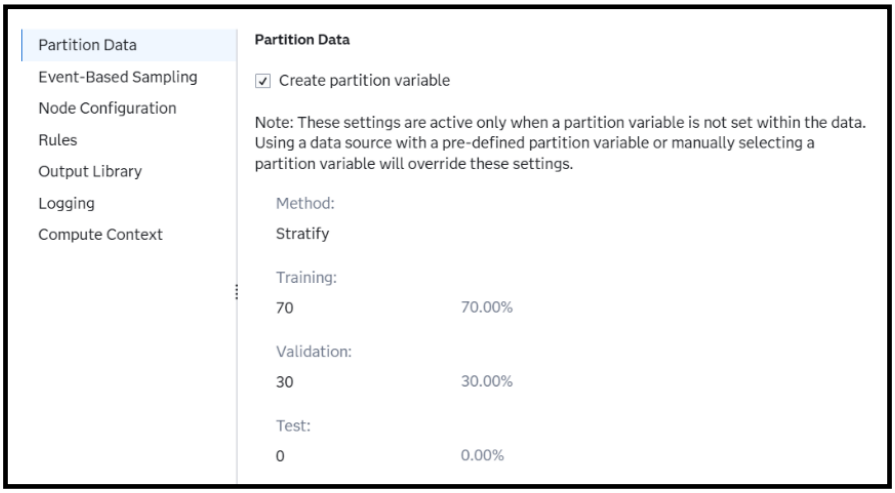

Figure 17. Partition Data Settings

To guarantee reliable model evaluation, stratified sampling was used to partition the dataset, with 70 percent (5,970 observations) of the data (the training data) and 30 percent (2,558 observations). Stratified sampling is used such that attack and non-attack records are also fairly balanced in training and validation subsets. This is especially crucial in intrusion detection issues, in which the issue of class imbalance can compromise model learning.

Event based Sampling

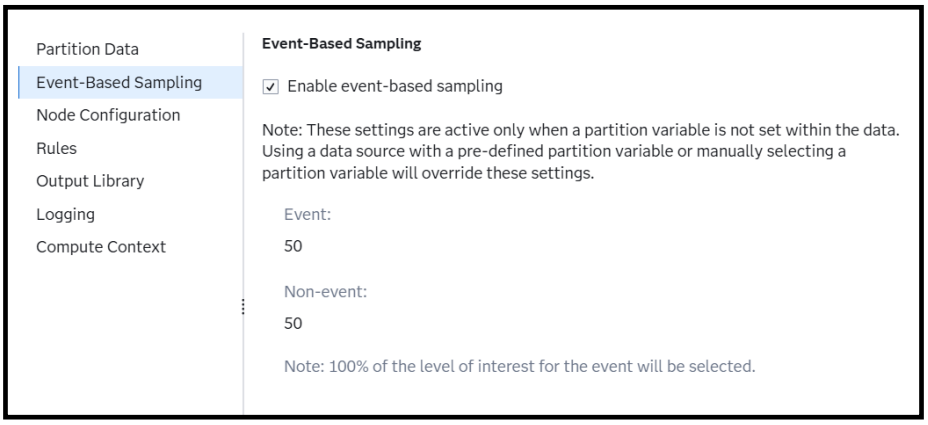

Figure 18. Event Based Settings

Moreover, event-based sampling was enabled at 50:50 ratio, a non-event and an event. This method assists in balancing both the attack events with the normal activity in the training process enabling the models to learn more effectively the patterns relating to intrusion detection.

3.2 Outlier Replacement

Outlier replacement was used to reduce the effects of the extreme values in the data set. Extreme values have the potential to distort data distribution when they are not addressed.

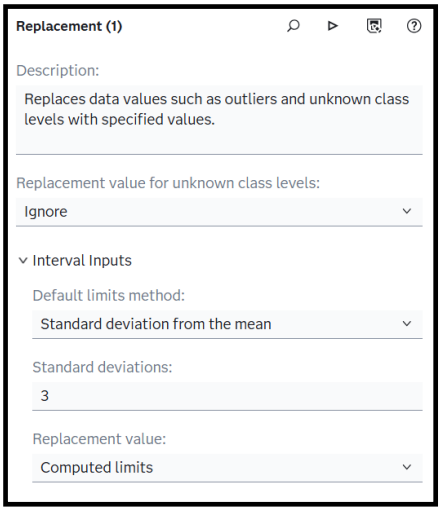

Figure 19. Replacement Settings for General Pipeline

Standard deviation from the mean method was utilised to identify outliers in the interval input variables using the replacement node. Values that were more than three standard deviations were considered outliers that were changed with the use of calculated limits. This method

assists in keeping data distribution constant by constraining the influence of extreme numbers and the models will be able to acquire more representative patterns of the data.

3.3 Variable Selection

Variable selection was used to select the most useful input variables in predicting the target variable attack_detected. This is to make sure that the model is less complex and that it can predict and predict more efficiently by eliminating the less informative predictors.

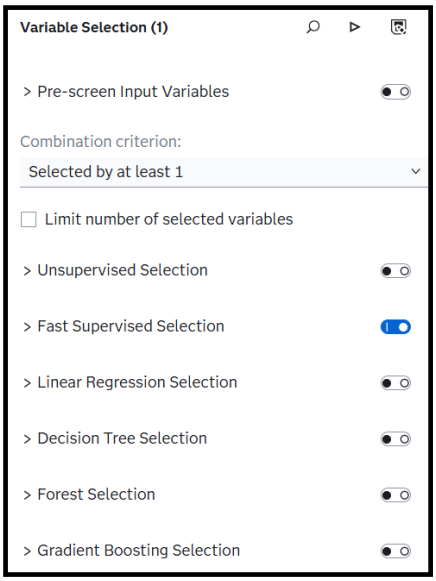

Figure 20. Variable Selection Settings foe General Pipeline

Fast supervised selection was applied in this work to measure the relationship between the input variables and the target variable. The variables that are selected in at least one of the methods were retained in modelling this process will only include those variables that are relevant and has the advantage of improving predictive performance and minimizes needless model complexity.

Upon preprocessing and selection, the final input variables that were to be used in modelling consisted of browser_type, encryption_used, failed_logins, ip_reputation_score, login_attempts, network_packet_size, and session_duration.

3.4 Transformation

Data transformation was used to enhance the distribution of numerical variables and improve model learning. Transformation of variables may assist in making the data skewness less and underscores the underlying trends.

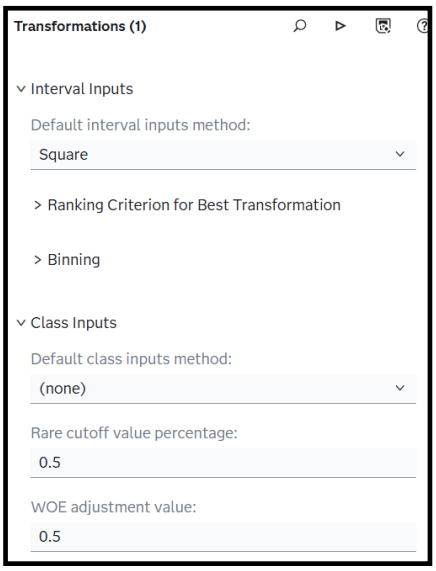

Figure 21. Transformation Settings for General Pipeline

Square transformations were used to transform several numerical variables to express possible nonlinear relationships between the predictors and the target variable. There was no transformation done on class input variables. Application of transformation exemplifies in enhancing the quality of the input variables, whereby there is a result of enhanced models in capturing the relationships that are defined with the target variable attack_detected. During the modelling process, some transformed variables that were not particularly important to the model were automatically discarded.

3.5 Imputation

The missing values in the dataset were dealt with using imputation. Incomplete data may adversely influence the process of training models, as well as decrease the reliability of predictions in case it is not carefully handled.

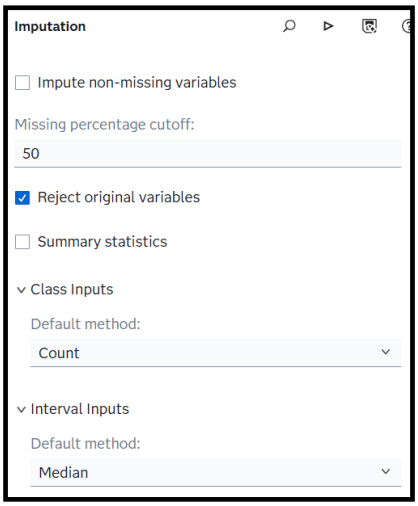

Figure 22. Imputation Settings for General Pipeline

In the current research, missing variable values of interval input variables were filled in with the median, whereas the class input variables were filled with the count method. Numerical variables were chosen as the median because it is not as sensitive to extreme numbers and has the effect of stabilising data distribution. This preprocessing process will make sure that the variables are of valid values so that the models can learn patterns pertaining to the target variable.

4. PIPELINE OF THE MODEL

4.1 GRADIENT BOOSTING

4.1.1 Model Construction and Optimization

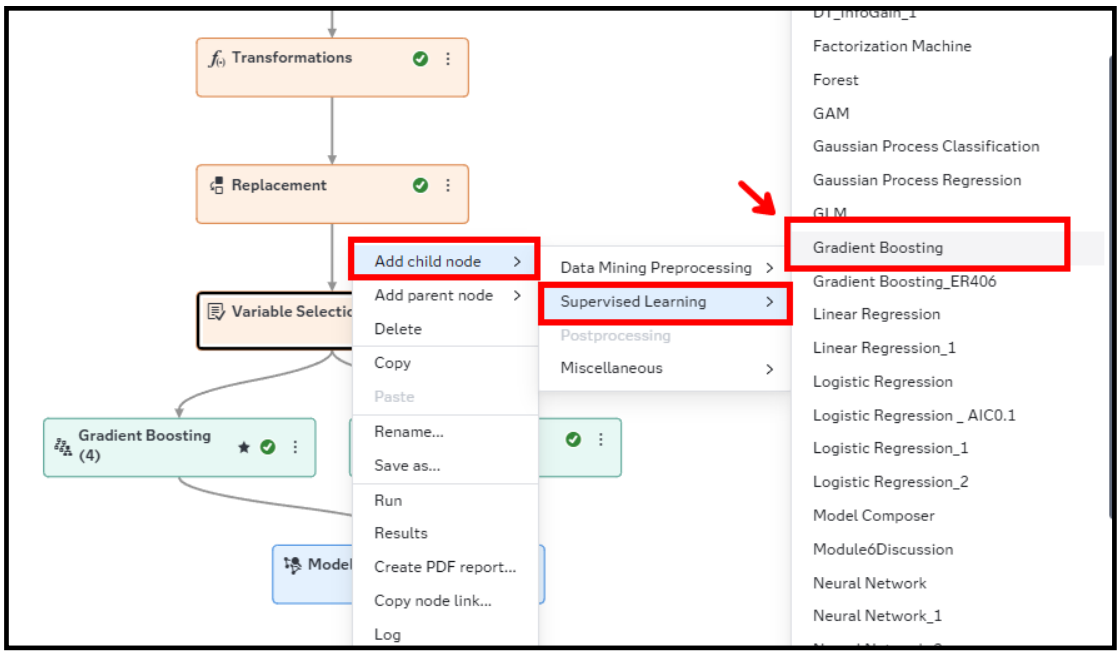

Figure 23. Adding the Gradient Boosting node

The node for gradient boosting model was placed after the preprocessing and variable selection stages so that the model was trained on cleaned, transformed, and reduced inputs. This ensured that boosting was applied to data that had already been prepared for stable and efficient modelling.

The following Gradient Boosting model tunings was applied in the Champion Model:

Basic Options

Figure 24. Basic Options setting for Gradient Boosting

The number of trees was set to 400 so that the model had enough boosting rounds to learn complex patterns, although it is also regulated using the early stopping method. Learning was slowed down to 0.03 in such a way that each tree only adds a small correction to the ensemble, which supports gradual learning but also reduces the risk of overfitting. The subsampling was configured to 0.8, i.e. only 80% of the training observations are being used in boosting, to introduce randomness that allows the model to generalize beyond the training set. L2 set to 3 will help to prevent any tree from dominating the prediction.

Tree Splitting Options

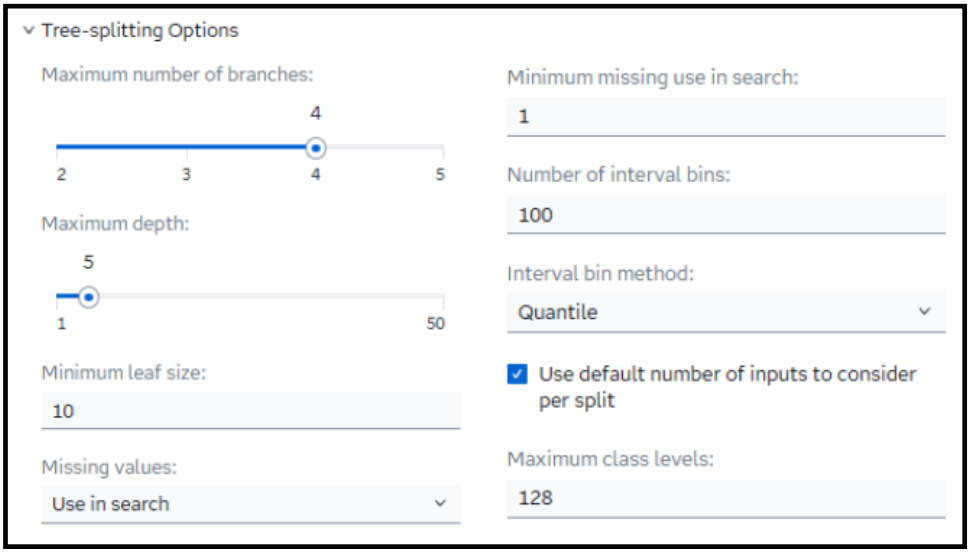

Figure 25. Tree Splitting Options setting for Gradient Boosting

A maximum of 4 branches and a maximum depth of 5 enables the model to learn the non- linear patterns but not to become too specific to the training data. The minimum leaf size was set to 10 to avoid unstable terminal nodes created by very small groups of observations. The bins used were increased to 100 to provide the model with higher precision in identifying split points among the continuous predictors.

Optimization – Early Stopping

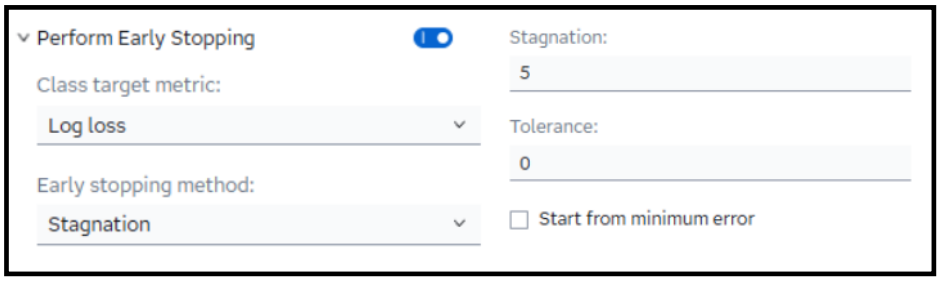

Figure 26. Optimization Setting for Gradient Boosting

Early stopping was enabled with Log loss for class target metric because it responds more sensitively to changes in predicted probabilities and thus a more appropriate indicator of when model starts to drift. The stopping method was set to stagnation with a patience of 5 to help in preventing the model from running far beyond its optimal point.

4.1.2 Model Validation

Error Plot

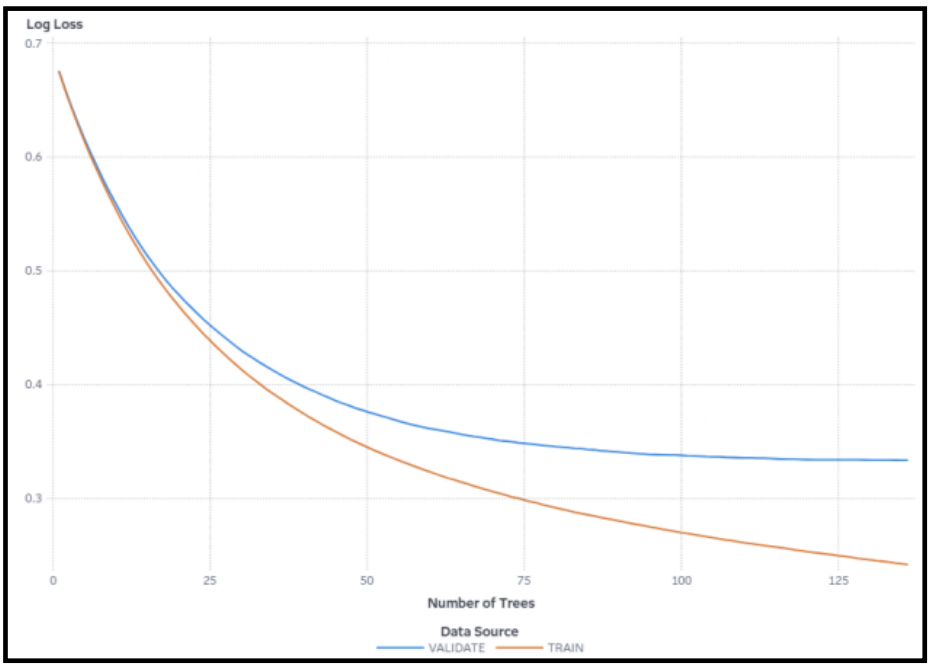

Figure 27. Log Loss plot - Gradient Boosting

The plot indicates that the model is learning effectively since both the training and validation log loss reduce with the increase in the number of trees. Nevertheless, at approximately 70 to 100 trees, the validation curve begins to flatten, as training curve remains to be improving very vigorously. This shows that early stopping was appropriate since it did not allow the model to run much further than it was best validated.

Fit Statistics Table

Data Role	Number of Observations	Misclassification Rate	Multi-Class Log Loss	Area Under ROC	Gini Coefficient
TRAIN	5,970	0.1136	0.2442	0.9816	0.9632
VALIDATE	2,558	0.1259	0.3373	0.8794	0.7587

Figure 28. Fit statistics - Gradient Boosting

The fit statistics show that the tuned Gradient Boosting model worked well on both partitions. The training misclassification rate was 0.1136, while the validation misclassification rate was

0.1259, giving a gap of only 0.0123 or 1.23 percentage points. This is way lower than the 10 percentage point mark that is commonly applied to indicate severe overfitting. The model also achieved training AUC of 0.9816 and validation AUC of 0.8794. Though the validation performance was quite good, the greater decrease in AUC and the rise in log loss between 0.2442 (training) and 0.3373 (validation) indicate that there was some overfitting in the probability estimation. Overall, the model generalized reasonably well, although it was more certain on the training data than on unseen data.

ROC Chart

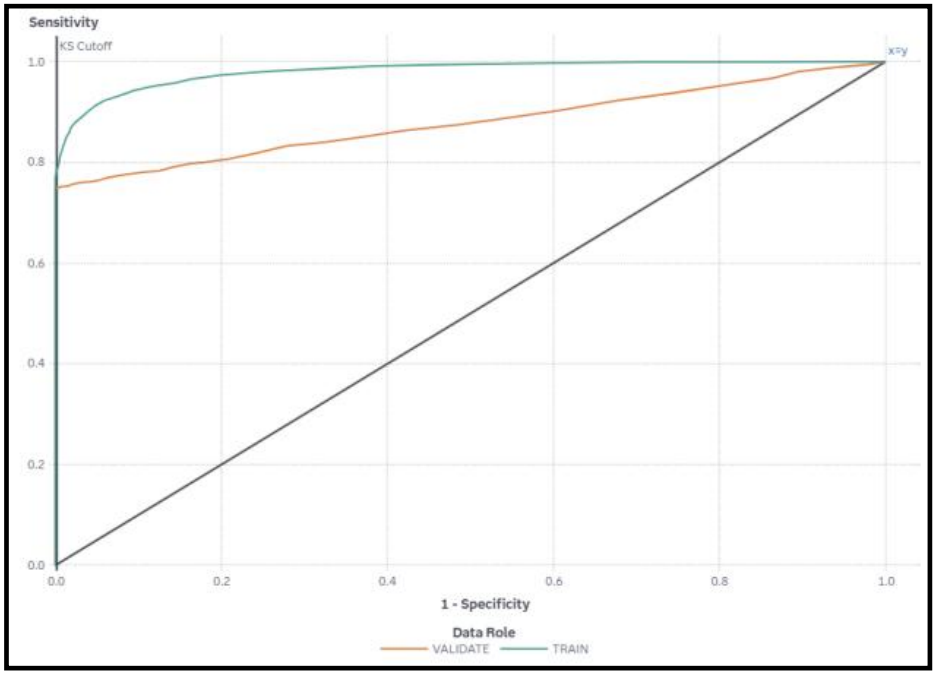

Figure 29. ROC Chart - Gradient Boosting

The ROC chart shows that the gradient boosting model has good discrimination ability because both curves lie clearly above the diagonal reference line, especially the validation curve. The training AUC of 0.9816 shows that the model has an excellent separation between intrusion and non-intrusion cases in the training partition whereas the validation AUC of 0.8794 confirms that the model continues to perform well on unseen data. The apparent discrepancy between the two curves indicates a certain level of overfitting, yet the validation curve is quite robust enough to regard the model as useful and reliable.

Event Classification Table

Cutoff	Cutoff Source	Value	Training Frequency	Validation Frequency	Training Percentage	Validation Percentage
0.2600	KS	True Positive	2,723	.	91.2228	.
0.2600	KS	False Negative	262	.	8.7772	.
0.2600	KS	True Negative	2,843	.	95.2429	.
0.2600	KS	False Positive	142	.	4.7571	.
0.5000	Default	True Positive	2,309	959	77.3534	74.9805
0.5000	Default	False Negative	676	320	22.6466	25.0195
0.5000	Default	True Negative	2,983	1,277	99.9330	99.8436
0.5000	Default	False Positive	2	2	0.0670	0.1564

Figure 30. Event Classification table - Gradient Boosting

The validation event classification table at the default cutoff of 0.50 indicates that the model accurately detected 959 intrusion events and 1,277 non-events with only 2 false positives. This gives very high validation specificity of 99.84%, which means the model is highly effective at avoiding false alarms. However, validation sensitivity was 74.98%, indicating that 320 actual intrusion events were still missed as false negatives. Thus, the model is robust in detecting normal traffic accurately, but it is less efficient when it comes to detecting all the positive intrusion cases.

4.1.3 Critical Interpretation

Surrogate Model Variable Importance

Variable Label	Variable Name	Relative Importance	Role	Variable Level
Replacement: Transformed failed_logins	REP_SQR_failed_logins	1	INPUT	INTERVAL
Replacement: Transformed login_attempts	REP_SQR_login_attempts	0.6008	INPUT	INTERVAL
Replacement: Transformed ip_reputation_score	REP_SQR_ip_reputation_score	0.4394	INPUT	INTERVAL
	browser_type	0.0893	INPUT	NOMINAL
Replacement: Transformed session_duration	REP_SQR_session_duration	0.0190	INPUT	INTERVAL
	encryption_used	0.0008	INPUT	NOMINAL

Figure 31. Surrogate Model Variable Importance Table - Gradient Boosting

The surrogate model shows that the gradient boosting model is explained mainly by failed logins, followed by login attempts and ip_reputation_score. This suggests the identification of attack events is driven mostly by suspicious login behaviour and security risk indicators. Conversely, browser type, session duration, and encryption have minimal contribution, so they do not provide a lot of explanatory information. A critical point is that the top predictors are

transformed variables, indicating that the square transformation improved signal quality of these variables and the model is capturing non-linear patterns.

PD Plot

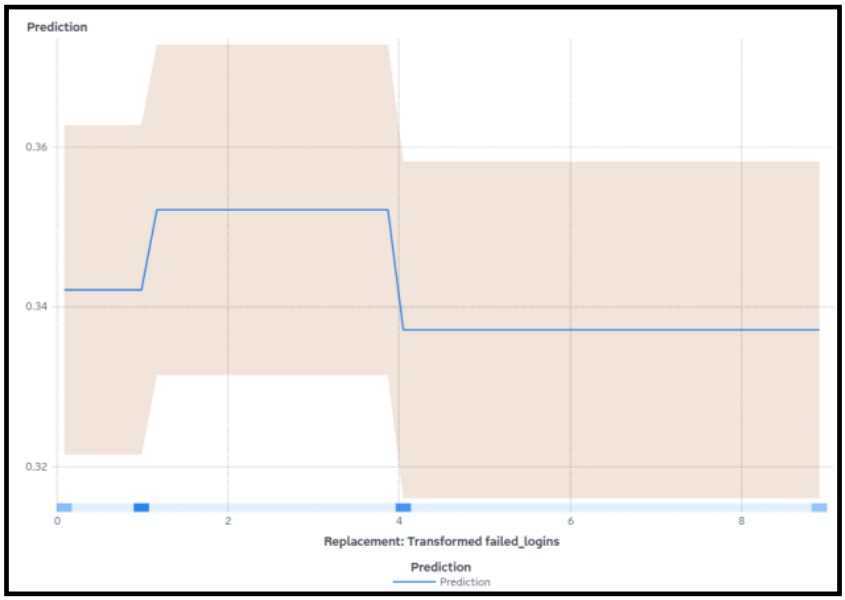

Figure 32. PD Plot - Gradient Boosting

The result of the partial dependence shows that failed_logins is an important predictor. The change in prediction is minimal over the range which indicates that the variable is significant primarily due to its interaction with other significant predictors. Therefore, though it is the first in importance, its influence is being worked out in interactions, not by a single marginal pattern.

HyperSHAP graph

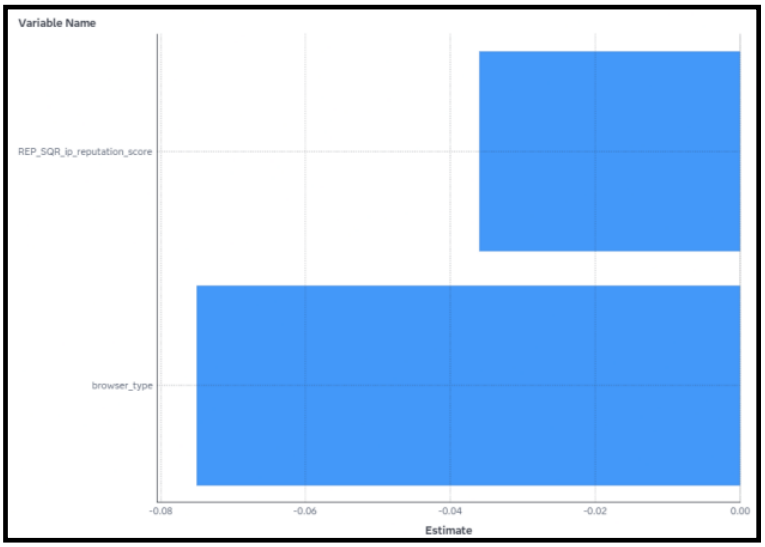

Figure 33. HyperSHAP Graph - Gradient Boosting

The HyperSHAP explanation supports the same interpretation at the individual level. The strongest local contributions come from failed logins, login attempts, and IP reputation score, which matches the global importance results. That makes the model easier to defend because the same variables matter both overall and for single predictions.

Closing Interpretation

The tuned Gradient Boosting model achieved strong validation performance with accuracy of 0.8741, F1 score of 0.8563, and AUC of 0.8794, however, the larger train and validate gap in AUC and log loss indicates that some probability level overfitting still exists.

4.2 RANDOM FOREST

4.2.1 Construction and Optimization

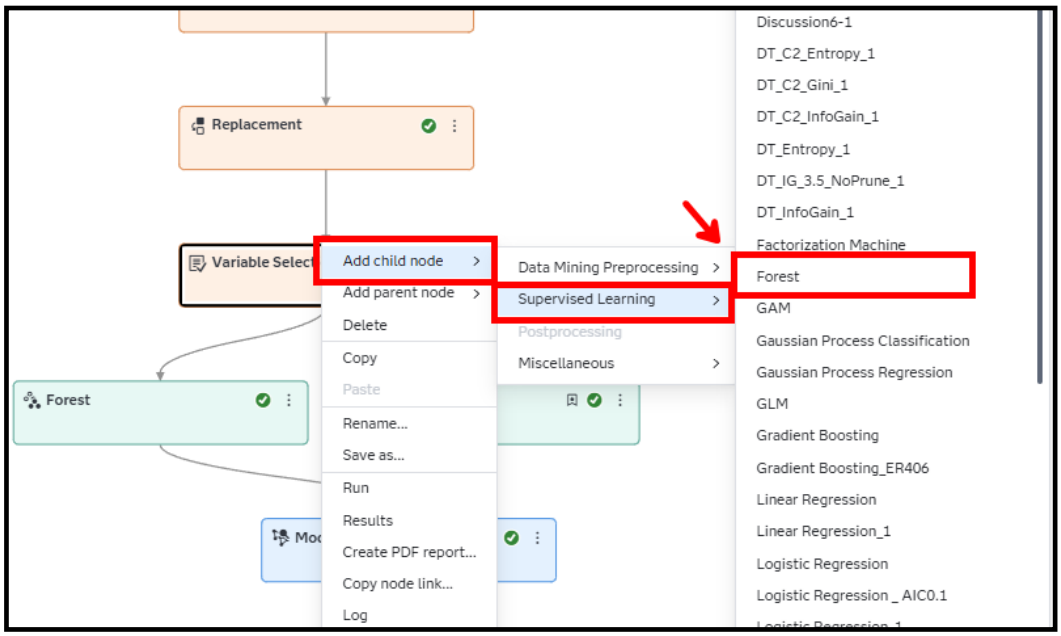

Figure 34. Adding Forest Node to pipeline

The Forest model was added after the preprocessing and variable selection stages so that the ensemble was built using cleaned, transformed and retained predictors. This placement ensured that the tree construction was based on inputs that had already been prepared for efficient modelling.

The following settings were tuned in the champion model:

Basic Options

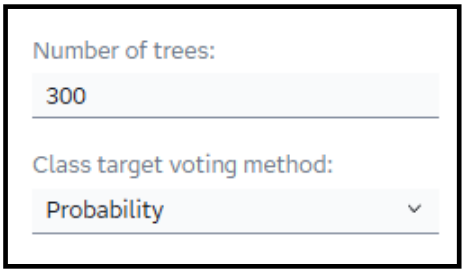

Figure 35. Basic Options setting – Forest

The number of trees was set to 300 to minimize the variance and enhance the stability of prediction amongst the ensemble. Voting method was to Probability that averages the predicted probabilities of all 300 trees yielding a more consistent probability outputs, significant to AUC and ROC parameters.

Tree Splitting Options

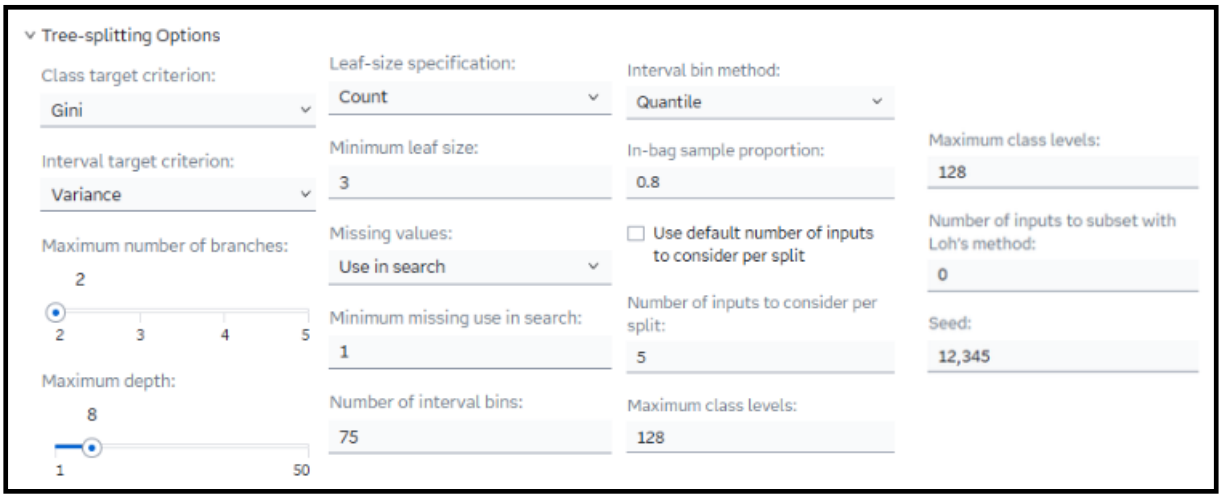

Figure 36. Tree Splitting Options – Forest

The choice of gini impurity as the splitting criterion was done because it is computationally-efficient and adapted to binary classification goals. Sampling 5 variables per split adds diversity to trees which is significant in random forest because it is part of what improves generalization. The tree size is controlled here to a maximum depth of 8 and maximum 2 branches so that the trees would not become unnecessarily complex. The leaf size of 3 in the model enabled it to represent local patterns without forming extremely unstable terminal nodes. The bag sample proportion was set to 0.8 so that each tree would be trained on a bootstrap sample of 80% of the data. Quantile binning with 75 bins was applied to continuous variables to handle their skewed distributions.

4.2.2 Model Validation

Fit Statistics Table

Data Role	Number of Observations	Misclassification Rate	Area Under ROC	Multi-Class Log Loss	Gini Coefficient
TRAIN	5,970	0.1188	0.9207	0.2928	0.8413
VALIDATE	2,558	0.1251	0.8794	0.3194	0.7588

Figure 37. Fit Statistics Table – Forest

The fit statistics indicate that the forest model generalised well to unseen data. The training misclassification rate was 0.1188, while the validation misclassification rate was 0.1251, giving a gap of only 0.0063 or 0.63 percentage points. This is a very small difference and indicates good generalisation with little evidence of serious overfitting. The model obtained training

AUC of 0.9207 and validation AUC of 0.8794, while log loss increased only from 0.2928 in training to 0.3194 in validation. These findings have indicated that the model was stable on the validation partition and generated reliable class separation.

ROC Chart

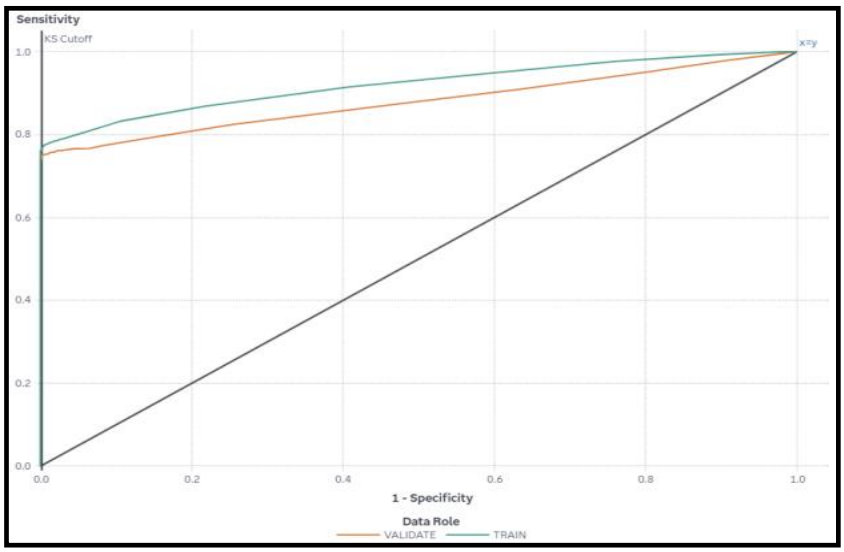

Figure 38. ROC Chart – Forest

The ROC chart shows that the forest model has good classification ability because both the training and validation curves are clearly above the diagonal line. The AUC of training and validation is 0.9207 and 0.8794 respectively, which means that the model is able to distinguish between attack and non-attack sessions in both partitions. The training curve still is greater than the validation curve, but the difference between them is not extreme, indicating that there is moderate overfitting. Overall, the validation ROC performance indicates that the model may continue to be highly useful in predicting unseen data.

Misclassification Rate

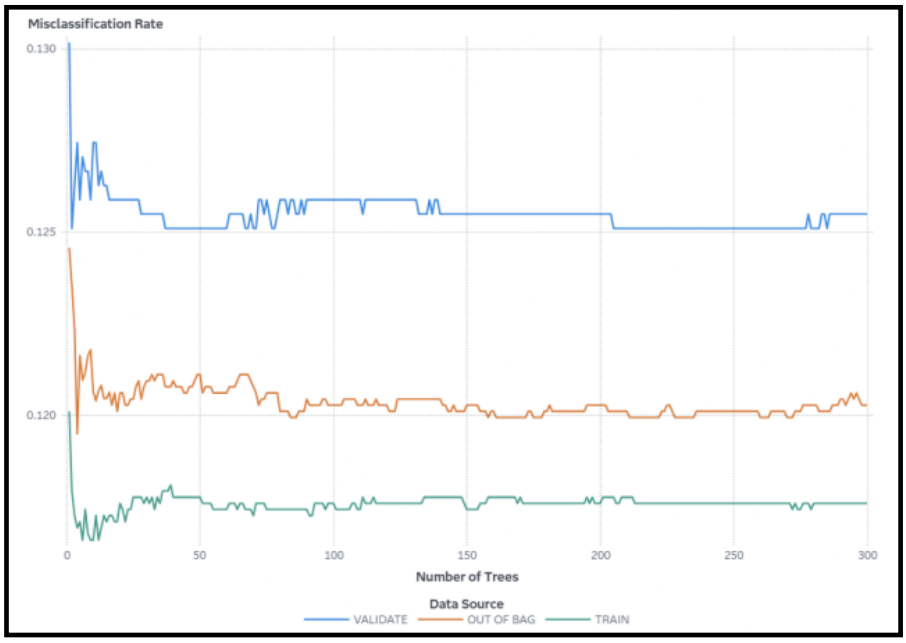

Figure 39. Misclassification Rate – Forest

The plot shows that the model improves very quickly in the first few trees, as the misclassification rate is decreasing rapidly initially. After that, all three curves become quite stable, so adding more trees gives only very small improvement. As the gap is small and seems to remain relatively constant, the model seems to be stable with minor overfitting rather than a serious generalization problem.

Event Classification Table

Cutoff	Cutoff Source	Value	Training Frequency	Validation Frequency	Training Percentage	Validation Percentage
0.3400	KS	True Positive	2,317	.	77.6214	.
0.3400	KS	False Negative	668	.	22.3786	.
0.3400	KS	True Negative	2,966	.	99.3635	.
0.3400	KS	False Positive	19	.	0.6365	.
0.4500	KS	True Positive	.	961	.	75.1368
0.4500	KS	False Negative	.	318	.	24.8632
0.4500	KS	True Negative	.	1,277	.	99.8436
0.4500	KS	False Positive	.	2	.	0.1564
0.5000	Default	True Positive	2,281	961	76.4154	75.1368
0.5000	Default	False Negative	704	318	23.5846	24.8632
0.5000	Default	True Negative	2,980	1,277	99.8325	99.8436
0.5000	Default	False Positive	5	2	0.1675	0.1564

Figure 40. Event Classification Table – Forest

At the default cutoff of 0.50, the validation event classification table shows that the model correctly identified 961 intrusion events and 1,277 non-events, with only 2 false positives. This

corresponds to validation sensitivity of 75.14% and specificity of 99.84%. The result shows that the model is highly effective at recognizing non-attack sessions and controlling false alarms. However, 318 actual intrusion events were still classified as false negatives, which means sensitivity remains the main area where further improvement would be desirable.

4.2.3 Critical Interpretation

Surrogate Model Variable Importance

Variable Label	Variable Name	Relative Importance	Role	Variable Level
Replacement: Transformed failed_logins	REP_SQR_failed_logins	1	INPUT	INTERVAL
Replacement: Transformed login_attempts	REP_SQR_login_attempts	0.6019	INPUT	INTERVAL
Replacement: Transformed ip_reputation_score	REP_SQR_ip_reputation_score	0.4518	INPUT	INTERVAL
	browser_type	0.0872	INPUT	NOMINAL
Replacement: Transformed session_duration	REP_SQR_session_duration	0.0146	INPUT	INTERVAL
	encryption_used	0.0006	INPUT	NOMINAL

Figure 41. Surrogate Model Variable Importance - Forest

The surrogate model shows that the forest model is explained mainly by failed logins, followed by login attempts and IP reputation score. This suggests the model relies mostly on suspicious authentication behaviour and risk related signals when predicting cyber intrusion. On the contrary, the level of contribution of browser type, session duration, and encryption is very little, so their explanatory value is limited. A critical point is that the strongest predictors are transformed variables, which indicates the model is capturing non linear patterns rather than simple direct relationships.

PD Plot

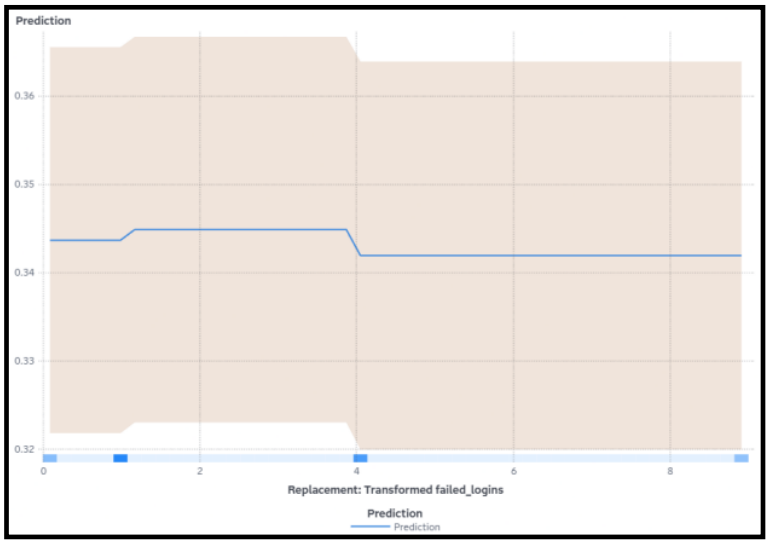

Figure 42. PD Plot – Forest

The partial dependence outcome for failed logins is relatively flat. That indicates that the variable has significance in general but its impact is being mediated largely by the interaction between the variable and the other predictors throughout the forest. So, its importance is real, but it is not expressed as one simple marginal trend.

HyperSHAP graph

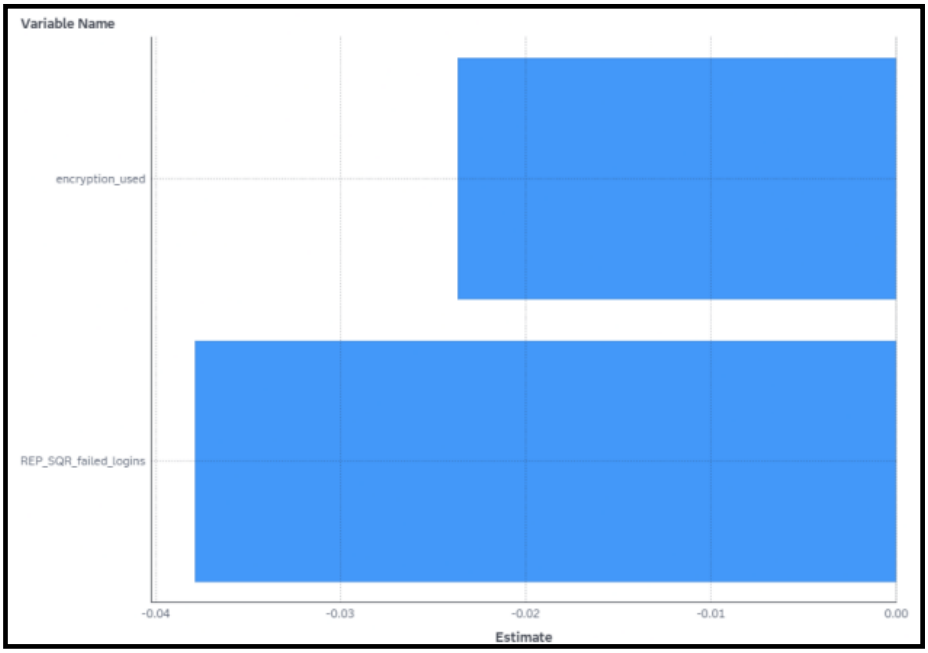

Figure 43. HyperSHAP Graph - Forest

Both variables shown in this graph have negative estimates, which means they are pushing the predicted probability downward. Among them, transformed failed_logins has the larger negative effect so it is the stronger contributor in lowering the prediction for this case. Encryption used also reduces the prediction, but less strongly. This finding illustrates the fact that the forest explanation can also be applied at the individual-case level and not just at the global level.

Closing interpretation

The tuned Forest model was found to have a validation accuracy of 0.8749, F1 score of 0.8573, and AUC of 0.8794, and it was slightly more stable than Gradient Boosting since it had a smaller train-validate gaps while maintaining equally strong discrimination.

5.0 Overall Model Comparison

Project Summary

Project Summary

The champion model for this project is Forest from the "Maimoona_RF" pipeline. The model was chosen based on the Accuracy for the Validate partition (0.87). 87.49% of the Validate partition was correctly classified using the Forest model. The five most important factors are Replacement: Transformed failed_logins, Replacement: Transformed login_attempts, Replacement: Transformed ip_reputation_score, browser_type, and Replacement: Transformed session_duration.

Project Target:	attack_detected	**Project Champion:**	Forest
Event Percentage:	50.0000%	**Created By:**	tp081109@mail.apu.edu.my
Pipelines:	8	**Modified:**	15 March 2026, 09:27:30

Figure 44. Project Summary

According to the project summary, the Forest model was selected as the most reliable overall model for the project since it had the strongest validation performance among all other models. This implies that it generalized more effectively to unseen data and offered the best balance between predictive accuracy and stability. The common predictors identified across all models also show that abnormal login behaviour, IP reputation, browser-related patterns, and session characteristics were always effective in differentiating between attack and normal activity. Overall, these findings justify the adoption of Forest model as the ultimate model of intrusion detection in this cyber study.

Most Common Variables

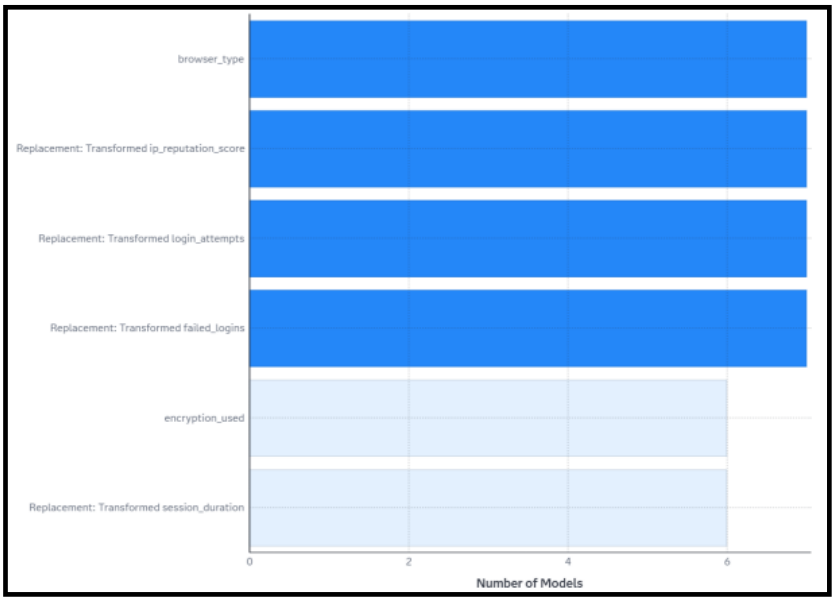

Figure 45. Most common variables selected across all models

As indicated in the given figure above, various predictors were chosen repeatedly when comparing the project models, which implies that they were always helpful in differentiating intrusion events from normal network activity. Specifically, login-related variables, IP reputation, and browser type appeared most frequently, which implied that the suspicious authentication behaviour and network credibility were the most reliable signals during the classification task. Multiple models also chose the session duration and encryption status, even though a bit less frequently. In general, it can be concluded that the models were able to align around a similar group of important predictors, which strengthens confidence in the stability and relevance of these variables.

Assessment for all Models – Champion Model

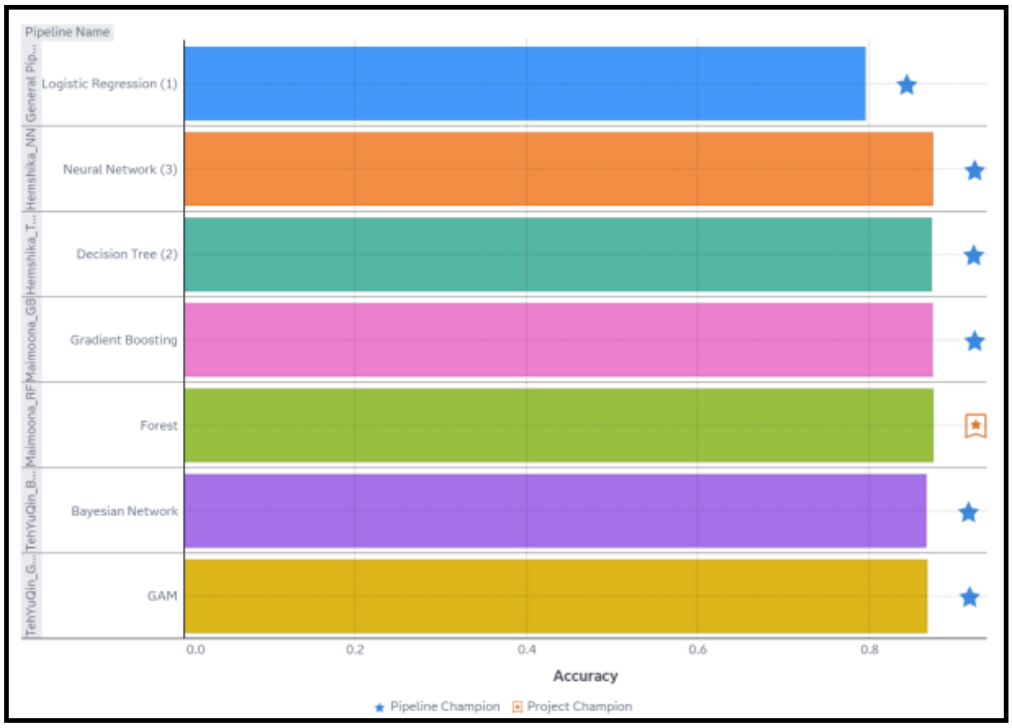

Figure 46. Assessment for all models

The bar chart provides the final comparison of all pipeline champions and confirms that the Forest model was the best overall solution for this project. While the strongest models performed closely, the Forest model achieved the top validation accuracy (0.8749) and was therefore selected as the final project champion ahead of Neural Network (0.8745) and Gradient Boosting (0.8741). This result indicates that the selected model offered the most reliable generalisation to unseen data among the competing approaches. Overall, the comparative assessment supports Forest as the most appropriate model for predicting cybersecurity intrusion events in this dataset.